

Numerical Methods for Ordinary Differential Equations

Math 104C — Advanced Numerical Analysis Final Project

Project repository: <https://github.com/elliottthull/Math104C-Advanced-Numerical-Analysis-Final-Project-I>

Introduction:

Ordinary Differential Equations (ODEs) arise naturally across science and engineering — from population dynamics and chemical reactions to electrical circuits and astrophysics. While many ODEs lack closed-form solutions, numerical methods allow us to approximate them to high accuracy. In this project, we investigate eight numerical solvers and analyze their behavior across several key problem settings.

This project considers the initial-value problem $y'(t) = f(t, y)$, $y(t_0) = y_0$ where we seek the function $y(t)$ on an interval $[a, b]$. Most such problems have no closed-form solution, so we approximate y at discrete points $t_i = a + ih$ using a step size h . The methods studied range from simple one-step schemes to higher-order Runge–Kutta and multistep predictor–corrector methods, and we compare their accuracy and error behavior as the step size is refined.

Numerical Methods:

We will examine methods spanning three levels of complexity. The Euler and Taylor 2nd-order methods are simple single-step schemes, with Taylor gaining accuracy as it uses derivative information to shrink that error. The Midpoint, Heun's, and RK4 methods are Runge-Kutta-type solvers that evaluate the right-hand side at intermediate points, with RK4 serving as the gold standard for single-step integration. Finally, Adams-Bashforth, Adams-Moulton, and the Predictor-Corrector are multi-step methods that achieve high accuracy with fewer function evaluations per step.

Euler's Method: The simplest one-step method. It advances the solution along the tangent line at each point:

$$w_{i+1} = w_i + h f(t_i, w_i).$$

First-order accurate, with global error $O(h)$. Easy to implement but requires very small steps for good accuracy.

****Taylor Method (order n):**** Improves on Euler by including higher derivative terms from the Taylor expansion of y :

$$w_{i+1} = w_i + h T^{(n)}(t_i, w_i), \quad T^{(n)} = f + \frac{h}{2} f' + \cdots + \frac{h^{n-1}}{n!} f^{(n-1)}.$$

Global error $O(h^n)$, but it requires computing analytic derivatives of f , which is often impractical.

Midpoint Method: A second-order Runge–Kutta method that evaluates the slope at the midpoint of the step:

$$w_{i+1} = w_i + h f(t_i + \frac{h}{2}, w_i + \frac{h}{2} f(t_i, w_i)).$$

Global error $O(h^2)$, achieving Taylor-order-2 accuracy without symbolic derivatives.

Heun's Method (Modified Euler): Another second-order Runge–Kutta method that averages the slopes at the start and predicted end of the interval:

$$w_{i+1} = w_i + \frac{h}{2} [f(t_i, w_i) + f(t_{i+1}, w_i + h f(t_i, w_i))].$$

Global error $O(h^2)$.

Runge–Kutta Method of Order 4 (RK4): The standard workhorse method, combining four slope evaluations per step:

$$w_{i+1} = w_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

$$k_1 = f(t_i, w_i), \quad k_2 = f(t_i + \frac{h}{2}, w_i + \frac{h}{2}k_1), \quad k_3 = f(t_i + \frac{h}{2}, w_i + \frac{h}{2}k_2), \quad k_4 = f(t_i + h, w_i + hk_3).$$

Global error $O(h^4)$, giving excellent accuracy with relatively large steps.

Adams–Bashforth Method (Explicit Multistep): Uses several previously computed points to extrapolate the next value, requiring only one new function evaluation per step. The fourth-order form is

$$w_{i+1} = w_i + \frac{h}{24}(55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}).$$

Efficient once started (a one-step method like RK4 supplies the initial points), with global error $O(h^4)$.

Adams–Moulton Method (Implicit Multistep): Similar to Adams–Bashforth but includes the unknown point f_{i+1} , making it implicit. The third-order form is

$$w_{i+1} = w_i + \frac{h}{24}(9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2}).$$

More accurate and stable than the explicit version of the same order, but requires solving for w_{i+1} .

Predictor–Corrector Method: Combines the two multistep schemes: Adams–Bashforth *predicts* a value w_{i+1}^* , which is substituted into Adams–Moulton to *correct* it. This pairs the efficiency of the explicit predictor with the accuracy and stability of the implicit corrector, achieving $O(h^4)$ accuracy with just two function evaluations per step.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

```
# Euler method
def euler(f, a, b, N, alpha):
    #create grid and time vector
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)

    w = np.zeros(N + 1)
    w[0] = alpha
    #loop for euler method
    for i in range(N):
        w[i + 1] = w[i] + h * f(t[i], w[i])
    return t, w
```

```

# Taylor 2nd-order method
def taylor2(f, f_prime, a, b, N, alpha):
    #create grid and time vector
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    #loop
    for i in range(N):
        #T_2 truncation error
        T_2 = f(t[i], w[i]) + (h / 2) * f_prime(t[i], w[i])

        w[i + 1] = w[i] + h * T_2
    return t, w

```

```

# Midpoint method
def midpoint(f, a, b, N, alpha):
    #create grid and time vector
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    #loops for midpoint
    for i in range(N):
        w[i + 1] = w[i] + h * f(t[i] + h / 2, w[i] + (h / 2) * f(t[i], w[i]))
    return t, w

```

```

# Heun's 3rd-order method
def heuns(f, a, b, N, alpha):
    #create grid
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        w[i + 1] = w[i] + (h / 4) * (
            f(t[i], w[i])
            + 3 * f(
                t[i] + (2 * h) / 3,
                w[i] + (2 * h / 3) * f(t[i] + h / 3, w[i] + (h / 3) * f(t[i], w[i]))
            )
        )
    return t, w

```

```

# Runge-Kutta 4th-order (RK4)
def RK4(f, a, b, N, alpha):
    #create grid
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    for i in range(N):
        #calculate 4 functions for next iteration update
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i + 1], w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    return t, w

```

```

# Adams-Bashforth 4-step explicit method
def adams_bashforth(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha
    # Bootstrap first 3 steps with RK4
    for i in range(3):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6
    # Adams-Bashforth 4-step formula
    for i in range(3, N):
        w[i + 1] = w[i] + (h / 24) * (
            55 * f(t[i], w[i])
            - 59 * f(t[i - 1], w[i - 1])
            + 37 * f(t[i - 2], w[i - 2])
            - 9 * f(t[i - 3], w[i - 3])
        )
    return t, w

```

```

# Adams-Moulton 3-step implicit method
def adams_moulton(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w[0] = alpha

    # 2 steps with RK4
    for i in range(2):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w[i] + (k1 + 2 * k2 + 2 * k3 + k4) / 6

    for i in range(2, N):
        # Predictor (Adams-Bashforth 3-step)
        w_pred = w[i] + (h / 12) * (
            23 * f(t[i], w[i])
            - 16 * f(t[i - 1], w[i - 1])
            + 5 * f(t[i - 2], w[i - 2])
        )
        # Corrector (Adams-Moulton 3-step)
        w[i + 1] = w[i] + (h / 24) * (
            9 * f(t[i + 1], w_pred)
            + 19 * f(t[i], w[i])
            - 5 * f(t[i - 1], w[i - 1])
            + f(t[i - 2], w[i - 2])
        )
    return t, w

```

```

def predictor_corrector(f, a, b, N, alpha):
    h = (b - a) / N
    t = np.linspace(a, b, N + 1)
    w = np.zeros(N + 1)
    w_p = np.zeros(N + 1)
    w[0] = w_p[0] = alpha

    # First 3 steps with RK4
    for i in range(3):
        k1 = h * f(t[i], w[i])
        k2 = h * f(t[i] + h / 2, w[i] + k1 / 2)
        k3 = h * f(t[i] + h / 2, w[i] + k2 / 2)
        k4 = h * f(t[i] + h, w[i] + k3)
        w[i + 1] = w_p[i + 1] = w[i] + (k1 + 2*k2 + 2*k3 + k4) / 6

    # Adams-Bashforth 4-step predictor + Adams-Moulton 4-step corrector
    for i in range(3, N):
        # Predictor
        w_p[i + 1] = w[i] + (h / 24) * (
            55 * f(t[i], w[i])
            - 59 * f(t[i - 1], w[i - 1])
            + 37 * f(t[i - 2], w[i - 2])
            - 9 * f(t[i - 3], w[i - 3])
        )
        # Corrector
        w[i + 1] = w[i] + (h / 24) * (
            9 * f(t[i + 1], w_p[i + 1])
            + 19 * f(t[i], w[i])
            - 5 * f(t[i - 1], w[i - 1])
            + f(t[i - 2], w[i - 2])
        )

    return t, w

```

Problem A)

```

# ODE:  $y' = y - t^2 + 1$ ,  $y(0) = 1$ 
# Exact solution:  $y(t) = (t+1)^2 - 0.5 * \exp(t)$ 

def f_1(t, y):
    return y - t**2 + 1

def f_1_p(t, y):
    return y - t**2 - 2*t + 1

def y_exact(t):
    return (t + 1)**2 - 0.5 * np.exp(t)

N_s = [10, 20, 40]

results = {}
for N in N_s:
    results[N] = {
        'euler': euler(f_1, 0, 2, N, alpha=0.5),
        'taylor': taylor2(f_1, f_1_p, 0, 2, N, alpha=0.5),
        'midpoint': midpoint(f_1, 0, 2, N, alpha=0.5),
        'heuns': heuns(f_1, 0, 2, N, alpha=0.5),
        'RK4': RK4(f_1, 0, 2, N, alpha=0.5),
        'adams_b': adams_bashforth(f_1, 0, 2, N, alpha=0.5),
        'adams_m': adams_moulten(f_1, 0, 2, N, alpha=0.5),
        'pred_cor': predictor_corrector(f_1, 0, 2, N, alpha=0.5),
    }

```

```

#graphing and numerical results
method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4',
                  'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

fig, axes = plt.subplots(2, 3, figsize=(16, 14))
fig.suptitle("ODE Method Comparison:  $y' = y - t^2 + 1$ ,  $y(0) = 0.5$ ", fontsize=14, fontweight='bold')

colors = ['#E24B4A', '#EF9F27', '#639922', '#1D9E75', '#185FA5', '#7F77DD', '#D4537E', '#854F0B']

for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)

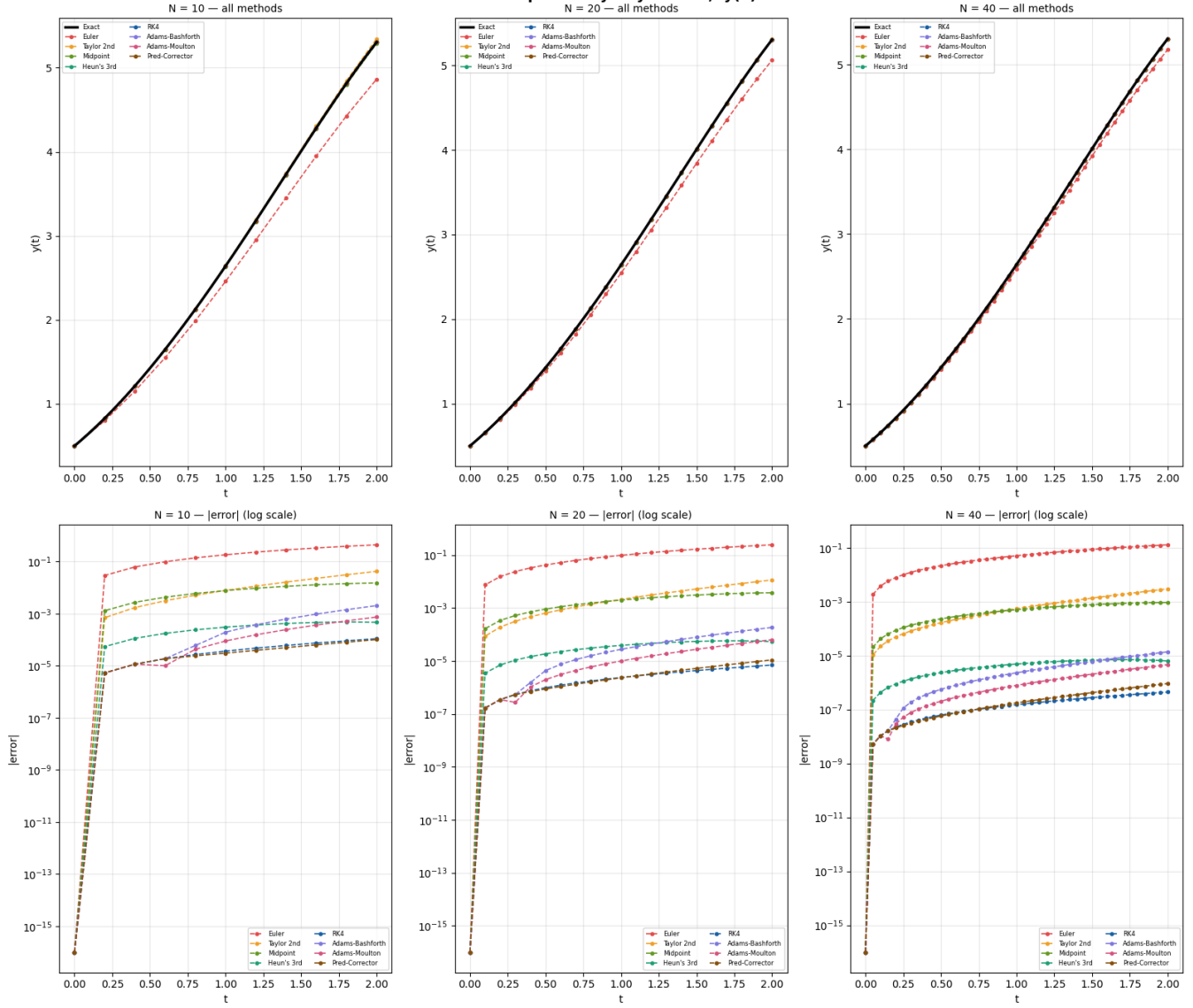
    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact(t_fine), 'k-', linewidth=2.5, label='Exact', zorder=10)
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewidth=1.2, label=label)
    ax_top.set_title(f'N = {N} - all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results[N][key]
        err = np.abs(w_m - y_exact(t_m))
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersize=3, linewidth=1.2, label=label)
    ax_mid.set_title(f'N = {N} - |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

ODE Method Comparison: $y' = y - t^2 + 1$, $y(0) = 0.5$




```

#create numerical tables for comparison
for N in N_s:
    t_vals = results[N]['euler'][0] # all methods share the same t
    exact_vals = y_exact(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        data[label] = w

    df = pd.DataFrame(data)

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        max_row[label] = np.max(np.abs(w - y_exact(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'  N = {N}  (h = {2/N:.4f})')
    print(f'{"="*60}')

    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))

```

```
=====
N = 10 (h = 0.2000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000
0.20000000	0.82929862	0.80000000	0.83000000	0.82800000	0.82924444	0.82929333	0.82929333	0.82929333	0.82929333
0.40000000	1.21408765	1.15200000	1.21580000	1.21136000	1.21397499	1.21407621	1.21407621	1.21407621	1.21407621
0.60000000	1.64894060	1.55040000	1.65207600	1.64465920	1.64876590	1.64892202	1.64892202	1.64895060	1.64892202
0.80000000	2.12722954	1.98848000	2.13233272	2.12128422	2.12699053	2.12720268	2.12728925	2.12727223	2.12720563
1.00000000	2.64085909	2.45817600	2.64864592	2.63316675	2.64055555	2.64082269	2.64105333	2.64094815	2.64082860
1.20000000	3.17994154	2.94981120	3.19134802	3.17046344	3.17957629	3.17989417	3.18031413	3.18009538	3.17990264
1.40000000	3.73240002	3.45177344	3.74864458	3.72116540	3.73198028	3.73234007	3.73301859	3.73264296	3.73235048
1.60000000	4.28348379	3.95012813	4.30614639	4.27062178	4.28302303	4.28340950	4.28444241	4.28384775	4.28342082
1.80000000	4.81517627	4.42815375	4.84629860	4.80095857	4.81469657	4.81508569	4.81659556	4.81570293	4.81509636
2.00000000	5.30547195	4.86578450	5.34768429	5.29036946	5.30500719	5.30536300	5.30750818	5.30621550	5.30537067
max error		0.43968745	0.04221234	0.01510249	0.00047969	0.00010895	0.00203623	0.00074355	0.00010128

```
=====
N = 20 (h = 0.1000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000
0.10000000	0.65741454	0.65000000	0.65750000	0.65725000	0.65741111	0.65741437	0.65741437	0.65741437	0.65741437
0.20000000	0.82929862	0.81400000	0.82948750	0.82896125	0.82929162	0.82929828	0.82929828	0.82929828	0.82929828
0.30000000	1.01507060	0.99140000	1.01538369	1.01455218	1.01505990	1.01507006	1.01507006	1.01507087	1.01507006
0.40000000	1.21408765	1.18154000	1.21454897	1.21338016	1.21407315	1.21408691	1.21408917	1.21408869	1.21408696
0.50000000	1.42563936	1.38369400	1.42627662	1.42473508	1.42562095	1.42563840	1.42564366	1.42564132	1.42563850
0.60000000	1.64894060	1.59706340	1.64978566	1.64783226	1.64891820	1.64893939	1.64894803	1.64894366	1.64893953
0.70000000	1.88312365	1.82076974	1.88421316	1.88180465	1.88309721	1.88312218	1.88313484	1.88312802	1.88312233
0.80000000	2.12722954	2.05384671	2.12860554	2.12569414	2.12719904	2.12722779	2.12724523	2.12723546	2.12722794
0.90000000	2.38019844	2.29523139	2.38190912	2.37844202	2.38016392	2.38019640	2.38021947	2.38020620	2.38019653
1.00000000	2.64085909	2.54375452	2.64295958	2.63887843	2.64082060	2.64085672	2.64088638	2.64086900	2.64085679
1.10000000	2.90791699	2.79812998	2.91047033	2.90571067	2.90787468	2.90791428	2.90795164	2.90792942	2.90791426
1.20000000	3.17994154	3.05694297	3.18301972	3.17751029	3.17989561	3.17993847	3.17998480	3.17995691	3.17993831
1.30000000	3.45535167	3.31863727	3.45903679	3.45269887	3.45530241	3.45534821	3.45540495	3.45537046	3.45534787
1.40000000	3.73240002	3.58150100	3.73678565	3.72953225	3.73234783	3.73239614	3.73246496	3.73242278	3.73239556
1.50000000	4.00915546	3.84365110	4.01434814	4.00608313	4.00910085	4.00915115	4.00923394	4.00918282	4.00915026
1.60000000	4.28348379	4.10301621	4.28960470	4.28022186	4.28342740	4.28347900	4.28357791	4.28351646	4.28347772
1.70000000	4.55302630	4.35731783	4.56021319	4.54959516	4.55296896	4.55302101	4.55313850	4.55306510	4.55301925
1.80000000	4.81517627	4.60404961	4.82358558	4.81160265	4.81511898	4.81517044	4.81530930	4.81522212	4.81516808
1.90000000	5.06705278	4.84045457	5.07686206	5.06337093	5.06699677	5.06704639	5.06720979	5.06710674	5.06704331
2.00000000	5.30547195	5.06350003	5.31688258	5.30172488	5.30541871	5.30546496	5.30565651	5.30553522	5.30546102
max error		0.24197192	0.01141063	0.00374707	0.00005734	0.00000699	0.00018456	0.00006327	0.00001093

=====

N = 40 (h = 0.0500)

=====

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000	0.50000000
0.05000000	0.57686445	0.57500000	0.57687500	0.57684375	0.57686424	0.57686445	0.57686445	0.57686445	0.57686445
0.10000000	0.65741454	0.65362500	0.65743672	0.65737262	0.65741411	0.65741453	0.65741453	0.65741453	0.65741453
0.15000000	0.74158288	0.73580625	0.74161785	0.74151921	0.74158222	0.74158286	0.74158286	0.74158289	0.74158286
0.20000000	0.82929862	0.82147156	0.82934764	0.82921270	0.82929773	0.82929860	0.82929866	0.82929865	0.82929860
0.25000000	0.92048729	0.91054514	0.92055171	0.92037860	0.92048617	0.92048726	0.92048740	0.92048734	0.92048727
0.30000000	1.01507060	1.00294740	1.01515186	1.01493863	1.01506924	1.01507056	1.01507078	1.01507067	1.01507057
0.35000000	1.11296623	1.09859477	1.11306589	1.11281048	1.11296463	1.11296618	1.11296649	1.11296633	1.11296619
0.40000000	1.21408765	1.19739951	1.21420739	1.21390764	1.21408581	1.21408760	1.21408801	1.21408778	1.21408761
0.45000000	1.31834391	1.29926948	1.31848552	1.31813916	1.31834182	1.31834385	1.31834436	1.31834407	1.31834386
0.50000000	1.42563936	1.40410796	1.42580478	1.42540942	1.42563703	1.42563930	1.42563992	1.42563957	1.42563931
0.55000000	1.53587349	1.51181335	1.53606477	1.53561790	1.53587091	1.53587342	1.53587417	1.53587373	1.53587343
0.60000000	1.64894060	1.62227902	1.64915997	1.64865894	1.64893777	1.64894052	1.64894140	1.64894088	1.64894053
0.65000000	1.76472959	1.73539297	1.76497942	1.76442146	1.76472650	1.76472950	1.76473052	1.76472992	1.76472950
0.70000000	1.88312365	1.85103762	1.88340649	1.88278869	1.88312031	1.88312355	1.88312473	1.88312403	1.88312355
0.75000000	2.00399999	1.96908950	2.00431857	2.00363786	2.00399640	2.00399989	2.00400124	2.00400043	2.00399989
0.80000000	2.12722954	2.08941898	2.12758677	2.12683992	2.12722569	2.12722942	2.12723096	2.12723003	2.12722942
0.85000000	2.25267657	2.21188993	2.25307559	2.25225922	2.25267248	2.25267645	2.25267819	2.25267713	2.25267644
0.90000000	2.38019844	2.33635942	2.38064259	2.37975313	2.38019410	2.38019831	2.38020027	2.38019907	2.38019830
0.95000000	2.50964517	2.46267739	2.51013802	2.50917173	2.50964058	2.50964503	2.50964722	2.50964587	2.50964501
1.00000000	2.64085909	2.59068626	2.64140447	2.64035741	2.64085425	2.64085894	2.64086138	2.64085987	2.64085891
1.05000000	2.77367444	2.72022058	2.77427645	2.77314447	2.77366937	2.77367428	2.77367699	2.77367531	2.77367425
1.10000000	2.90791699	2.85110660	2.90858000	2.90735875	2.90791168	2.90791682	2.90791982	2.90791795	2.90791678
1.15000000	3.04340355	2.98316193	3.04413222	3.04281714	3.04339802	3.04340336	3.04340668	3.04340461	3.04340331
1.20000000	3.17994154	3.11619503	3.18074087	3.17932714	3.17993580	3.17994134	3.17994501	3.17994272	3.17994129
1.25000000	3.31732852	3.25000478	3.31820384	3.31668641	3.31732257	3.31732831	3.31733234	3.31732982	3.31732825
1.30000000	3.45535167	3.38438002	3.45630866	3.45468221	3.45534552	3.45535145	3.45535587	3.45535309	3.45535136
1.35000000	3.59378723	3.51909902	3.59483198	3.59309092	3.59378091	3.59378700	3.59379185	3.59378879	3.59378691
1.40000000	3.73240002	3.65392897	3.73353900	3.73167746	3.73239353	3.73239977	3.73240507	3.73240172	3.73239966
1.45000000	3.87094274	3.78862542	3.87218287	3.87019468	3.87093611	3.87094248	3.87094827	3.87094461	3.87094235
1.50000000	4.00915546	3.92293169	4.01050412	4.00838278	4.00914871	4.00915519	4.00916150	4.00915750	4.00915504
1.55000000	4.14676491	4.05657828	4.14822995	4.14596865	4.14675805	4.14676462	4.14677149	4.14676713	4.14676445
1.60000000	4.28348379	4.18928219	4.28507361	4.28266517	4.28347685	4.28348348	4.28349096	4.28348620	4.28348329
1.65000000	4.41901009	4.32074630	4.42073364	4.41817051	4.41900309	4.41900977	4.41901789	4.41901271	4.41900955
1.70000000	4.55302630	4.45065862	4.55489311	4.55216737	4.55301929	4.55302597	4.55303478	4.55302915	4.55302572
1.75000000	4.68519866	4.57869155	4.68721888	4.68432220	4.68519166	4.68519831	4.68520786	4.68520175	4.68519804
1.80000000	4.81517627	4.70450113	4.81736072	4.81428433	4.81516932	4.81517590	4.81518625	4.81517961	4.81517559
1.85000000	4.94259024	4.82772618	4.94495046	4.94168516	4.94258338	4.94258985	4.94260105	4.94259386	4.94258951
1.90000000	5.06705278	4.94798749	5.06960105	5.06613715	5.06704606	5.06705237	5.06706448	5.06705670	5.06705199
1.95000000	5.18815621	5.06488687	5.19090560	5.18723292	5.18814968	5.18815579	5.18816887	5.18816045	5.18815536
2.00000000	5.30547195	5.17800621	5.30843639	5.30454424	5.30546566	5.30547151	5.30548563	5.30547653	5.30547104
max error		0.12746574	0.00296444	0.00092771	0.00000701	0.00000044	0.00001368	0.00000458	0.00000091

```

#error tables for comparison
import pandas as pd

method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4',
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals = results[N]['euler'][0]
    exact_vals = y_exact(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals}
    for key, label in zip(method_keys, method_labels):
        _, w = results[N][key]
        data[label] = np.abs(w - exact_vals)

    df = pd.DataFrame(data)

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table | h = {h} (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()

```

Error table | $h = 0.2$ ($N = 10$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.500000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.20000000	0.829299	2.93e-02	7.01e-04	1.30e-03	5.42e-05	5.29e-06	5.29e-06	5.29e-06	5.29e-06
0.40000000	1.214088	6.21e-02	1.71e-03	2.73e-03	1.13e-04	1.14e-05	1.14e-05	1.14e-05	1.14e-05
0.60000000	1.648941	9.85e-02	3.14e-03	4.28e-03	1.75e-04	1.86e-05	1.86e-05	1.00e-05	1.86e-05
0.80000000	2.127230	1.39e-01	5.10e-03	5.95e-03	2.39e-04	2.69e-05	5.97e-05	4.27e-05	2.39e-05
1.00000000	2.640859	1.83e-01	7.79e-03	7.69e-03	3.04e-04	3.64e-05	1.94e-04	8.91e-05	3.05e-05
1.20000000	3.179942	2.30e-01	1.14e-02	9.48e-03	3.65e-04	4.74e-05	3.73e-04	1.54e-04	3.89e-05
1.40000000	3.732400	2.81e-01	1.62e-02	1.12e-02	4.20e-04	5.99e-05	6.19e-04	2.43e-04	4.95e-05
1.60000000	4.283484	3.33e-01	2.27e-02	1.29e-02	4.61e-04	7.43e-05	9.59e-04	3.64e-04	6.30e-05
1.80000000	4.815176	3.87e-01	3.11e-02	1.42e-02	4.80e-04	9.06e-05	1.42e-03	5.27e-04	7.99e-05
2.00000000	5.305472	4.40e-01	4.22e-02	1.51e-02	4.65e-04	1.09e-04	2.04e-03	7.44e-04	1.01e-04

Error table | $h = 0.1$ ($N = 20$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.500000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.10000000	0.657415	7.41e-03	8.55e-05	1.65e-04	3.43e-06	1.66e-07	1.66e-07	1.66e-07	1.66e-07
0.20000000	0.829299	1.53e-02	1.89e-04	3.37e-04	7.00e-06	3.45e-07	3.45e-07	3.45e-07	3.45e-07
0.30000000	1.015071	2.37e-02	3.13e-04	5.18e-04	1.07e-05	5.38e-07	5.38e-07	2.72e-07	5.38e-07
0.40000000	1.214088	3.25e-02	4.61e-04	7.07e-04	1.45e-05	7.45e-07	1.52e-06	1.04e-06	6.90e-07
0.50000000	1.425639	4.19e-02	6.37e-04	9.04e-04	1.84e-05	9.69e-07	4.30e-06	1.96e-06	8.67e-07
0.60000000	1.648941	5.19e-02	8.45e-04	1.11e-03	2.24e-05	1.21e-06	7.43e-06	3.06e-06	1.07e-06
0.70000000	1.883124	6.24e-02	1.09e-03	1.32e-03	2.64e-05	1.47e-06	1.12e-05	4.37e-06	1.32e-06
0.80000000	2.127230	7.34e-02	1.38e-03	1.54e-03	3.05e-05	1.75e-06	1.57e-05	5.93e-06	1.60e-06
0.90000000	2.380198	8.50e-02	1.71e-03	1.76e-03	3.45e-05	2.04e-06	2.10e-05	7.76e-06	1.92e-06
1.00000000	2.640859	9.71e-02	2.10e-03	1.98e-03	3.85e-05	2.36e-06	2.73e-05	9.91e-06	2.29e-06
1.10000000	2.907917	1.10e-01	2.55e-03	2.21e-03	4.23e-05	2.70e-06	3.47e-05	1.24e-05	2.73e-06
1.20000000	3.179942	1.23e-01	3.08e-03	2.43e-03	4.59e-05	3.07e-06	4.33e-05	1.54e-05	3.22e-06
1.30000000	3.455352	1.37e-01	3.69e-03	2.65e-03	4.93e-05	3.46e-06	5.33e-05	1.88e-05	3.80e-06
1.40000000	3.732400	1.51e-01	4.39e-03	2.87e-03	5.22e-05	3.88e-06	6.49e-05	2.28e-05	4.45e-06
1.50000000	4.009155	1.66e-01	5.19e-03	3.07e-03	5.46e-05	4.32e-06	7.85e-05	2.74e-05	5.21e-06
1.60000000	4.283484	1.80e-01	6.12e-03	3.26e-03	5.64e-05	4.79e-06	9.41e-05	3.27e-05	6.07e-06
1.70000000	4.553026	1.96e-01	7.19e-03	3.43e-03	5.73e-05	5.29e-06	1.12e-04	3.88e-05	7.06e-06
1.80000000	4.815176	2.11e-01	8.41e-03	3.57e-03	5.73e-05	5.83e-06	1.33e-04	4.59e-05	8.18e-06
1.90000000	5.067053	2.27e-01	9.81e-03	3.68e-03	5.60e-05	6.39e-06	1.57e-04	5.40e-05	9.47e-06
2.00000000	5.305472	2.42e-01	1.14e-02	3.75e-03	5.32e-05	6.99e-06	1.85e-04	6.33e-05	1.09e-05

Error table | $h = 0.05$ ($N = 40$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.500000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.05000000	0.576864	1.86e-03	1.05e-05	2.07e-05	2.16e-07	5.20e-09	5.20e-09	5.20e-09	5.20e-09
0.10000000	0.657415	3.79e-03	2.22e-05	4.19e-05	4.36e-07	1.06e-08	1.06e-08	1.06e-08	1.06e-08
0.15000000	0.741583	5.78e-03	3.50e-05	6.37e-05	6.60e-07	1.62e-08	1.62e-08	7.94e-09	1.62e-08
0.20000000	0.829299	7.83e-03	4.90e-05	8.59e-05	8.88e-07	2.20e-08	4.29e-08	2.86e-08	2.07e-08
0.25000000	0.920487	9.94e-03	6.44e-05	1.09e-04	1.12e-06	2.80e-08	1.12e-07	5.14e-08	2.55e-08
0.30000000	1.015071	1.21e-02	8.13e-05	1.32e-04	1.36e-06	3.43e-08	1.85e-07	7.63e-08	3.08e-08
0.35000000	1.112966	1.44e-02	9.97e-05	1.56e-04	1.60e-06	4.08e-08	2.66e-07	1.04e-07	3.66e-08
0.40000000	1.214088	1.67e-02	1.20e-04	1.80e-04	1.84e-06	4.75e-08	3.55e-07	1.34e-07	4.29e-08
0.45000000	1.318344	1.91e-02	1.42e-04	2.05e-04	2.08e-06	5.45e-08	4.52e-07	1.67e-07	4.98e-08
0.50000000	1.425639	2.15e-02	1.65e-04	2.30e-04	2.33e-06	6.18e-08	5.58e-07	2.02e-07	5.72e-08
0.55000000	1.535873	2.41e-02	1.91e-04	2.56e-04	2.58e-06	6.93e-08	6.74e-07	2.41e-07	6.53e-08
0.60000000	1.648941	2.67e-02	2.19e-04	2.82e-04	2.83e-06	7.71e-08	8.01e-07	2.84e-07	7.40e-08
0.65000000	1.764730	2.93e-02	2.50e-04	3.08e-04	3.08e-06	8.52e-08	9.39e-07	3.30e-07	8.35e-08
0.70000000	1.883124	3.21e-02	2.83e-04	3.35e-04	3.34e-06	9.35e-08	1.09e-06	3.81e-07	9.37e-08
0.75000000	2.004000	3.49e-02	3.19e-04	3.62e-04	3.59e-06	1.02e-07	1.25e-06	4.35e-07	1.05e-07
0.80000000	2.127230	3.78e-02	3.57e-04	3.90e-04	3.84e-06	1.11e-07	1.43e-06	4.94e-07	1.17e-07
0.85000000	2.252677	4.08e-02	3.99e-04	4.17e-04	4.10e-06	1.20e-07	1.62e-06	5.58e-07	1.30e-07
0.90000000	2.380198	4.38e-02	4.44e-04	4.45e-04	4.35e-06	1.30e-07	1.83e-06	6.28e-07	1.44e-07
0.95000000	2.509645	4.70e-02	4.93e-04	4.73e-04	4.59e-06	1.40e-07	2.05e-06	7.03e-07	1.59e-07
1.00000000	2.640859	5.02e-02	5.45e-04	5.02e-04	4.84e-06	1.50e-07	2.29e-06	7.84e-07	1.75e-07
1.05000000	2.773674	5.35e-02	6.02e-04	5.30e-04	5.07e-06	1.61e-07	2.55e-06	8.71e-07	1.92e-07
1.10000000	2.907917	5.68e-02	6.63e-04	5.58e-04	5.30e-06	1.72e-07	2.83e-06	9.66e-07	2.11e-07
1.15000000	3.043404	6.02e-02	7.29e-04	5.86e-04	5.53e-06	1.83e-07	3.14e-06	1.07e-06	2.31e-07
1.20000000	3.179942	6.37e-02	7.99e-04	6.14e-04	5.74e-06	1.95e-07	3.47e-06	1.18e-06	2.53e-07
1.25000000	3.317329	6.73e-02	8.75e-04	6.42e-04	5.95e-06	2.07e-07	3.82e-06	1.30e-06	2.76e-07
1.30000000	3.455352	7.10e-02	9.57e-04	6.69e-04	6.14e-06	2.20e-07	4.20e-06	1.42e-06	3.01e-07
1.35000000	3.593787	7.47e-02	1.04e-03	6.96e-04	6.32e-06	2.33e-07	4.61e-06	1.56e-06	3.28e-07
1.40000000	3.732400	7.85e-02	1.14e-03	7.23e-04	6.49e-06	2.46e-07	5.05e-06	1.71e-06	3.57e-07
1.45000000	3.870943	8.23e-02	1.24e-03	7.48e-04	6.63e-06	2.60e-07	5.53e-06	1.86e-06	3.88e-07
1.50000000	4.009155	8.62e-02	1.35e-03	7.73e-04	6.76e-06	2.74e-07	6.04e-06	2.03e-06	4.21e-07
1.55000000	4.146765	9.02e-02	1.47e-03	7.96e-04	6.86e-06	2.89e-07	6.58e-06	2.22e-06	4.57e-07
1.60000000	4.283484	9.42e-02	1.59e-03	8.19e-04	6.94e-06	3.04e-07	7.17e-06	2.41e-06	4.95e-07
1.65000000	4.419010	9.83e-02	1.72e-03	8.40e-04	6.99e-06	3.20e-07	7.80e-06	2.62e-06	5.36e-07
1.70000000	4.553026	1.02e-01	1.87e-03	8.59e-04	7.01e-06	3.36e-07	8.48e-06	2.85e-06	5.79e-07
1.75000000	4.685199	1.07e-01	2.02e-03	8.76e-04	7.00e-06	3.52e-07	9.20e-06	3.09e-06	6.26e-07
1.80000000	4.815176	1.11e-01	2.18e-03	8.92e-04	6.95e-06	3.69e-07	9.98e-06	3.35e-06	6.76e-07
1.85000000	4.942590	1.15e-01	2.36e-03	9.05e-04	6.86e-06	3.87e-07	1.08e-05	3.62e-06	7.29e-07
1.90000000	5.067053	1.19e-01	2.55e-03	9.16e-04	6.72e-06	4.05e-07	1.17e-05	3.92e-06	7.87e-07
1.95000000	5.188156	1.23e-01	2.75e-03	9.23e-04	6.53e-06	4.23e-07	1.27e-05	4.24e-06	8.48e-07
2.00000000	5.305472	1.27e-01	2.96e-03	9.28e-04	6.29e-06	4.42e-07	1.37e-05	4.58e-06	9.13e-07

Problem A uses the ODE $y' = y - t^2 + 1$ with exact solution $y(t) = (t + 1)^2 - \frac{1}{2}e^t$. Because the solution is smooth and non-stiff, it provides an excellent test of convergence behavior. The numerical results clearly reflect the theoretical orders of the methods. At $t=2$, Euler's error decreases from approximately $4.40 * e^{-1}$ for $N=10$ to $2.42 * e^{-1}$ for $N=20$, and then to roughly $1.27 * e^{-1}$ for $N=40$, demonstrating first-order convergence. The second-order methods (Taylor and Midpoint) show much faster improvement. Taylor's error falls from approximately $4.22 * e^{-02}$ to $2.96 * e^{-03}$, while the Midpoint method decreases from $1.51 * e^{-02}$ to roughly $9.28 * e^{-04}$. Higher-order methods perform dramatically better as expected. Heun's method, RK4, Adams-Moulton, and the Predictor-Corrector method all produce errors several orders of magnitude smaller than Euler's method. In the plots, the RK4 and multistep solutions are nearly indistinguishable from the exact solution, even with relatively large step sizes. As the step size decreases, the errors consistently decrease at rates predicted by the theoretical convergence orders, confirming the expected relationship between local truncation error and global error.

Problem B

```
def f_2(t, y):
    #problem
    return 2*y

def y_exact_2(t):
    #exact solutio
    return np.exp(2*t)

def f_2_p(t,y):
    #derivative
    return 2 * np.exp(2*t)

#step sizes
N_s = [10, 20, 40]

results_2 = {}
for N in N_s:
    results_2[N] = {
        'euler': euler(f_2, 0, 2, N, alpha=1),
        'taylor': taylor2(f_2, f_2_p, 0, 2, N, alpha=1),
        'midpoint': midpoint(f_2, 0, 2, N, alpha=1),
        'heuns': heuns(f_2, 0, 2, N, alpha=1),
        'RK4': RK4(f_2, 0, 2, N, alpha=1),
        'adams_b': adams_bashforth(f_2, 0, 2, N, alpha=1),
        'adams_m': adams_moulten(f_2, 0, 2, N, alpha=1),
        'pred_cor': predictor_corrector(f_2, 0, 2, N, alpha=1),
    }
```

```

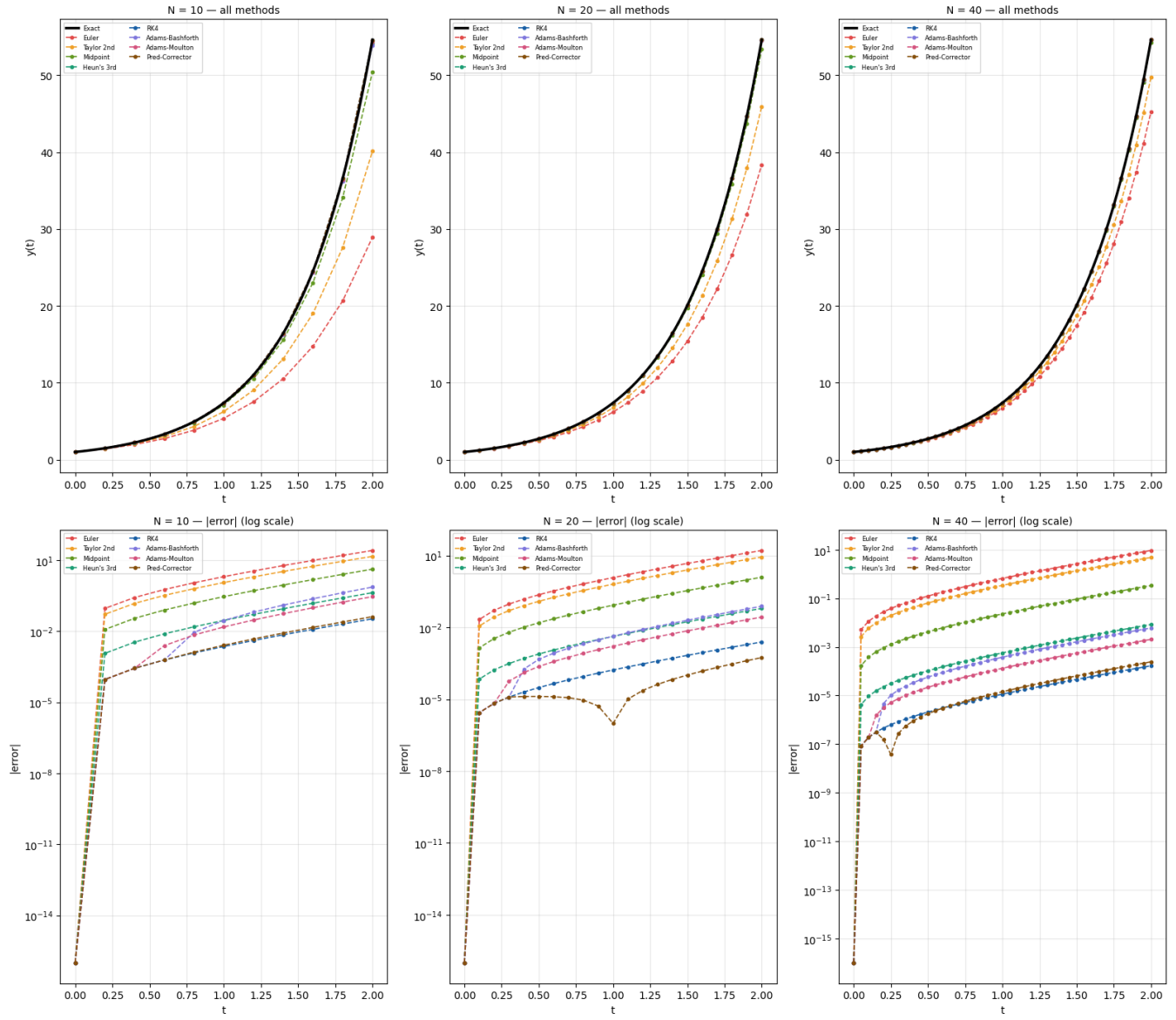
#plotting
fig, axes = plt.subplots(2, 3, figsize=(16, 14))

for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)
    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact_2(t_fine), 'k-', linewidth=2.5, label='Exact', zorder=10)
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_2[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewidth=1.2, label=label)
    ax_top.set_title(f'N = {N} - all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_2[N][key]
        err = np.abs(w_m - y_exact_2(t_m)) # ← fixed
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersize=3, linewidth=1.2, label=label)
    ax_mid.set_title(f'N = {N} - |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```

#numerical tables and solution
for N in N_s:

    t_vals = results_2[N]['euler'][0]    # all methods share the same t
    exact_vals_2 = y_exact_2(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals_2}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
        data[label] = w

    df = pd.DataFrame(data)

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
        max_row[label] = np.max(np.abs(w - y_exact_2(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'   N = {N}   (h = {2/N:.4f})')
    print(f'{"="*60}')

    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))

```

```
=====
N = 10 (h = 0.2000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
0.20000000	1.49182470	1.40000000	1.44000000	1.48000000	1.49066667	1.49173333	1.49173333	1.49173333	1.49173333
0.40000000	2.22554093	1.96000000	2.07567299	2.19040000	2.22208711	2.22526834	2.22526834	2.22526834	2.22526834
0.60000000	3.32011692	2.74400000	2.99496382	3.24179200	3.31239119	3.31950696	3.31950696	3.31764964	3.31950696
0.80000000	4.95303242	3.84160000	4.32575403	4.79785216	4.93767113	4.95181918	4.94411002	4.94611091	4.95172385
1.00000000	7.38905610	5.37824000	6.25417693	7.10082120	7.36042176	7.38679372	7.36051784	7.37393550	7.38649862
1.20000000	11.02317638	7.52953600	9.05140995	10.50921537	10.97193538	11.01912642	10.95919005	10.99347860	11.01846292
1.40000000	16.44464677	10.54135040	13.11290098	15.55363875	16.35549833	16.43759819	16.31821352	16.38969714	16.43627512
1.60000000	24.53253020	14.75789056	19.01584725	23.01938535	24.38059618	24.52051314	24.29740853	24.43468346	24.51804179
1.80000000	36.59823444	20.66104678	27.60348736	34.06869032	36.34334204	36.57806681	36.17787924	36.42860207	36.57363785
2.00000000	54.59815003	28.92546550	40.10881168	50.42166167	54.17580854	54.56472152	53.86750332	54.30981136	54.55700735
max error		25.67268454	14.48933836	4.17648836	0.42234150	0.03342851	0.73064672	0.28833867	0.04114268

```
=====
N = 20 (h = 0.1000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
0.10000000	1.22140276	1.20000000	1.21000000	1.22000000	1.22133333	1.22140000	1.22140000	1.22140000	1.22140000
0.20000000	1.49182470	1.44000000	1.46421403	1.48840000	1.49165511	1.49181796	1.49181796	1.49181796	1.49181796
0.30000000	1.82211880	1.72800000	1.77197508	1.81584800	1.82180811	1.82210646	1.82210646	1.82206225	1.82210646
0.40000000	2.22554093	2.07360000	2.14459128	2.21533456	2.22503497	2.22552083	2.22535975	2.22541104	2.22552788
0.50000000	2.71828183	2.48832000	2.59576495	2.70270816	2.71750938	2.71825114	2.71781950	2.71804936	2.71826869
0.60000000	3.32011692	2.98598400	3.14210076	3.29730396	3.31898479	3.32007194	3.31928137	3.31974285	3.32010416
0.70000000	4.05519997	3.58318080	3.80372208	4.02271083	4.05358675	4.05513587	4.05385202	4.05463299	4.05518841
0.80000000	4.95303242	4.29981696	4.60501850	4.90770721	4.95078062	4.95294295	4.95097988	4.95220547	4.95302323
0.90000000	6.04964746	5.15978035	5.57555252	5.98740280	6.04655340	6.04952451	6.04664372	6.04847320	6.04964225
1.00000000	7.38905610	6.19173642	6.75115950	7.30463142	7.38485722	7.38888924	7.38478191	7.38742128	7.38905708
1.10000000	9.02501350	7.43008371	8.17528196	8.91165033	9.01937228	9.02478932	9.01905369	9.02277175	9.02502367
1.20000000	11.02317638	8.91610045	9.90058848	10.87221340	11.01566001	11.02287768	11.01499385	11.02013911	11.02319976
1.30000000	13.46373804	10.69932054	11.99093794	13.26410035	13.45379276	13.46334279	13.45264077	13.45966287	13.46377998
1.40000000	16.44464677	12.83918465	14.52376291	16.18220242	16.43156556	16.44412689	16.42974540	16.43922302	16.44471435
1.50000000	20.08553692	15.40702157	17.59296196	19.74228696	20.06841873	20.08485658	20.06569107	20.07836721	20.08563944
1.60000000	24.53253020	18.48842589	21.31240972	24.08559009	24.51022875	24.53164383	24.50628102	24.52310727	24.53267980
1.70000000	29.96410005	22.18611107	25.82021697	29.38441990	29.93515938	29.96294977	29.92958515	29.95177766	29.96431257
1.80000000	36.59823444	26.62333328	31.28390137	35.84899228	36.56080799	36.59674685	36.55308068	36.58219063	36.59853041
1.90000000	44.70118449	31.94799994	37.90666398	43.73577059	44.65293349	44.69926660	44.64237311	44.68037545	44.70159044
2.00000000	54.59815003	38.33759992	45.93500863	53.35764012	54.53611610	54.59568423	54.52184713	54.57125219	54.59870015
max error		16.26055011	8.66314141	1.24050992	0.06203394	0.00246581	0.07630290	0.02689785	0.00055011

=====

N = 40 (h = 0.0500)

=====

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000	1.00000000
0.05000000	1.10517092	1.10000000	1.10250000	1.10500000	1.10516667	1.10517083	1.10517083	1.10517083	1.10517083
0.10000000	1.22140276	1.21000000	1.21551293	1.22102500	1.22139336	1.22140257	1.22140257	1.22140257	1.22140257
0.15000000	1.34985881	1.33100000	1.34011773	1.34923262	1.34984323	1.34985850	1.34985850	1.34985729	1.34985850
0.20000000	1.49182470	1.46410000	1.47750415	1.49090205	1.49180174	1.49182424	1.49182011	1.49182155	1.49182454
0.25000000	1.64872127	1.61051000	1.62898412	1.64744677	1.64868956	1.64872064	1.64871099	1.64871617	1.64872131
0.30000000	1.82211880	1.77156100	1.79600434	1.82042868	1.82207674	1.82211796	1.82210207	1.82211137	1.82211907
0.35000000	2.01375271	1.94871710	1.98016007	2.01157369	2.01369848	2.01375163	2.01372825	2.01374251	2.01375327
0.40000000	2.22554093	2.14358881	2.18321046	2.22278892	2.22547244	2.22553956	2.22550724	2.22552747	2.22554183
0.45000000	2.45960311	2.35794769	2.40709536	2.45618176	2.45951796	2.45960141	2.45955853	2.45958582	2.45960442
0.50000000	2.71828183	2.59374246	2.65395390	2.71408085	2.71817726	2.71827974	2.71822444	2.71826005	2.71828362
0.55000000	3.00416602	2.85311671	2.92614499	2.99905934	3.00403890	3.00416349	3.00409363	3.00413900	3.00416838
0.60000000	3.32011692	3.13842838	3.22626991	3.31396057	3.31996366	3.32011387	3.32002699	3.32008379	3.32011995
0.65000000	3.66929667	3.45227121	3.55719719	3.66192643	3.66911317	3.66929301	3.66918632	3.66925644	3.66930048
0.70000000	4.05519997	3.79749834	3.92209015	4.04642870	4.05498158	4.05519561	4.05506590	4.05515152	4.05520470
0.75000000	4.48168907	4.17724817	4.32443717	4.47130371	4.48143047	4.48168392	4.48152751	4.48163112	4.48169487
0.80000000	4.95303242	4.59497299	4.76808511	4.94079060	4.95272758	4.95302635	4.95283908	4.95296351	4.95303947
0.85000000	5.47394739	5.05447028	5.25727620	5.45957362	5.47358943	5.47394026	5.47371736	5.47386585	5.47395587
0.90000000	6.04964746	5.55991731	5.79668869	6.03282885	6.04922858	6.04963911	6.04937517	6.04955139	6.04965760
0.95000000	6.68589444	6.11590904	6.39148167	6.66627588	6.68540579	6.68588470	6.68557354	6.68578169	6.68590650
1.00000000	7.38905610	6.72749995	7.04734458	7.36623484	7.38848763	7.38904477	7.38867937	7.38892422	7.38907036
1.05000000	8.16616991	7.40024994	7.77055168	8.13968950	8.16551025	8.16615676	8.16572917	8.16601614	8.16618672
1.10000000	9.02501350	8.14027494	8.56802227	8.99435690	9.02424974	9.02499828	9.02449944	9.02483468	9.02503322
1.15000000	9.97418245	8.95430243	9.44738703	9.93876437	9.97330000	9.97416486	9.97358454	9.97397502	9.97420552
1.20000000	11.02317638	9.84973268	10.41706119	10.98233463	11.02215872	11.02315610	11.02248265	11.02293629	11.02320327
1.25000000	12.18249396	10.83470594	11.48632525	12.13547977	12.18132241	12.18247061	12.18169088	12.18221663	12.18252523
1.30000000	13.46373804	11.91817654	12.66541401	13.40970514	13.46239149	13.46371119	13.46281028	13.46341830	13.46377430
1.35000000	14.87973172	13.10999419	13.96561475	14.81772418	14.87818633	14.87970092	14.87866195	14.87936373	14.87977370
1.40000000	16.44464677	14.42099361	15.39937556	16.37358522	16.44287559	16.44461146	16.44341536	16.44422390	16.44469525
1.45000000	18.17414537	15.86309297	16.98042473	18.09281167	18.17211800	18.17410496	18.17273016	18.17366016	18.17420126
1.50000000	20.08553692	17.44940227	18.72390257	19.99255690	20.08321908	20.08549072	20.08391288	20.08498093	20.08560125
1.55000000	22.19795128	19.19434250	20.64650667	22.09177537	22.19530429	22.19789852	22.19609013	22.19731498	22.19802519
1.60000000	24.53253020	21.11377675	22.76665221	24.41141178	24.52951045	24.53247000	24.53040003	24.53180285	24.53261500
1.65000000	27.11263892	23.22515442	25.10464876	26.97461002	27.10919730	27.11257032	27.11020373	27.11180842	27.11273609
1.70000000	29.96410005	25.54766986	27.68289523	29.80694407	29.96018122	29.96402193	29.96131924	29.96315273	29.96421125
1.75000000	33.11545196	28.10243685	30.52609501	32.93667320	33.11099361	33.11536309	33.11227977	33.11437245	33.11557907
1.80000000	36.59823444	30.91268053	33.66149314	36.39502389	36.59316644	36.59813342	36.59461931	36.59700541	36.59837958
1.85000000	40.44730436	34.00394859	37.11913804	40.21650140	40.44154778	40.44718961	40.44318821	40.44590629	40.44746990
1.90000000	44.70118449	37.40434344	40.93217010	44.43923404	44.69465055	44.70105424	44.69650192	44.69959543	44.70137313
1.95000000	49.40244911	41.14477779	45.13714007	49.10535362	49.39503797	49.40230137	49.39712649	49.40064433	49.40266386
2.00000000	54.59815003	45.25925557	49.77436020	54.26141575	54.58974946	54.59798257	54.59210455	54.59610176	54.59839432
max error		9.33889446	4.82378983	0.33673429	0.00840057	0.00016746	0.00604549	0.00204828	0.00024429

```

#error tables and analysis
method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4',
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals = results_2[N]['euler'][0]
    exact_vals_2 = y_exact_2(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals_2}
    for key, label in zip(method_keys, method_labels):
        _, w = results_2[N][key]
        data[label] = np.abs(w - exact_vals_2)

    df = pd.DataFrame(data)

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table | h = {h} (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()

```


Error table | $h = 0.2$ ($N = 10$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	1.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.20000000	1.491825	9.18e-02	5.18e-02	1.18e-02	1.16e-03	9.14e-05	9.14e-05	9.14e-05	9.14e-05
0.40000000	2.225541	2.66e-01	1.50e-01	3.51e-02	3.45e-03	2.73e-04	2.73e-04	2.73e-04	2.73e-04
0.60000000	3.320117	5.76e-01	3.25e-01	7.83e-02	7.73e-03	6.10e-04	6.10e-04	2.47e-03	6.10e-04
0.80000000	4.953032	1.11e+00	6.27e-01	1.55e-01	1.54e-02	1.21e-03	8.92e-03	6.92e-03	1.31e-03
1.00000000	7.389056	2.01e+00	1.13e+00	2.88e-01	2.86e-02	2.26e-03	2.85e-02	1.51e-02	2.56e-03
1.20000000	11.023176	3.49e+00	1.97e+00	5.14e-01	5.12e-02	4.05e-03	6.40e-02	2.97e-02	4.71e-03
1.40000000	16.444647	5.90e+00	3.33e+00	8.91e-01	8.91e-02	7.05e-03	1.26e-01	5.49e-02	8.37e-03
1.60000000	24.532530	9.77e+00	5.52e+00	1.51e+00	1.52e-01	1.20e-02	2.35e-01	9.78e-02	1.45e-02
1.80000000	36.598234	1.59e+01	8.99e+00	2.53e+00	2.55e-01	2.02e-02	4.20e-01	1.70e-01	2.46e-02
2.00000000	54.598150	2.57e+01	1.45e+01	4.18e+00	4.22e-01	3.34e-02	7.31e-01	2.88e-01	4.11e-02

Error table | $h = 0.1$ ($N = 20$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	1.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.10000000	1.221403	2.14e-02	1.14e-02	1.40e-03	6.94e-05	2.76e-06	2.76e-06	2.76e-06	2.76e-06
0.20000000	1.491825	5.18e-02	2.76e-02	3.42e-03	1.70e-04	6.74e-06	6.74e-06	6.74e-06	6.74e-06
0.30000000	1.822119	9.41e-02	5.01e-02	6.27e-03	3.11e-04	1.23e-05	1.23e-05	5.66e-05	1.23e-05
0.40000000	2.225541	1.52e-01	8.09e-02	1.02e-02	5.06e-04	2.01e-05	1.81e-04	1.30e-04	1.31e-05
0.50000000	2.718282	2.30e-01	1.23e-01	1.56e-02	7.72e-04	3.07e-05	4.62e-04	2.32e-04	1.31e-05
0.60000000	3.320117	3.34e-01	1.78e-01	2.28e-02	1.13e-03	4.50e-05	8.36e-04	3.74e-04	1.28e-05
0.70000000	4.055200	4.72e-01	2.51e-01	3.25e-02	1.61e-03	6.41e-05	1.35e-03	5.67e-04	1.16e-05
0.80000000	4.953032	6.53e-01	3.48e-01	4.53e-02	2.25e-03	8.95e-05	2.05e-03	8.27e-04	9.19e-06
0.90000000	6.049647	8.90e-01	4.74e-01	6.22e-02	3.09e-03	1.23e-04	3.00e-03	1.17e-03	5.22e-06
1.00000000	7.389056	1.20e+00	6.38e-01	8.44e-02	4.20e-03	1.67e-04	4.27e-03	1.63e-03	9.77e-07
1.10000000	9.025013	1.59e+00	8.50e-01	1.13e-01	5.64e-03	2.24e-04	5.96e-03	2.24e-03	1.02e-05
1.20000000	11.023176	2.11e+00	1.12e+00	1.51e-01	7.52e-03	2.99e-04	8.18e-03	3.04e-03	2.34e-05
1.30000000	13.463738	2.76e+00	1.47e+00	2.00e-01	9.95e-03	3.95e-04	1.11e-02	4.08e-03	4.19e-05
1.40000000	16.444647	3.61e+00	1.92e+00	2.62e-01	1.31e-02	5.20e-04	1.49e-02	5.42e-03	6.76e-05
1.50000000	20.085537	4.68e+00	2.49e+00	3.43e-01	1.71e-02	6.80e-04	1.98e-02	7.17e-03	1.03e-04
1.60000000	24.532530	6.04e+00	3.22e+00	4.47e-01	2.23e-02	8.86e-04	2.62e-02	9.42e-03	1.50e-04
1.70000000	29.964100	7.78e+00	4.14e+00	5.80e-01	2.89e-02	1.15e-03	3.45e-02	1.23e-02	2.13e-04
1.80000000	36.598234	9.97e+00	5.31e+00	7.49e-01	3.74e-02	1.49e-03	4.52e-02	1.60e-02	2.96e-04
1.90000000	44.701184	1.28e+01	6.79e+00	9.65e-01	4.83e-02	1.92e-03	5.88e-02	2.08e-02	4.06e-04
2.00000000	54.598150	1.63e+01	8.66e+00	1.24e+00	6.20e-02	2.47e-03	7.63e-02	2.69e-02	5.50e-04

Error table | $h = 0.05$ ($N = 40$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	1.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.05000000	1.105171	5.17e-03	2.67e-03	1.71e-04	4.25e-06	8.47e-08	8.47e-08	8.47e-08	8.47e-08
0.10000000	1.221403	1.14e-02	5.89e-03	3.78e-04	9.40e-06	1.87e-07	1.87e-07	1.87e-07	1.87e-07
0.15000000	1.349859	1.89e-02	9.74e-03	6.26e-04	1.56e-05	3.11e-07	3.11e-07	1.52e-06	3.11e-07
0.20000000	1.491825	2.77e-02	1.43e-02	9.23e-04	2.30e-05	4.58e-07	4.59e-06	3.15e-06	1.57e-07
0.25000000	1.648721	3.82e-02	1.97e-02	1.27e-03	3.17e-05	6.32e-07	1.03e-05	5.10e-06	3.76e-08
0.30000000	1.822119	5.06e-02	2.61e-02	1.69e-03	4.21e-05	8.38e-07	1.67e-05	7.43e-06	2.73e-07
0.35000000	2.013753	6.50e-02	3.36e-02	2.18e-03	5.42e-05	1.08e-06	2.45e-05	1.02e-05	5.58e-07
0.40000000	2.225541	8.20e-02	4.23e-02	2.75e-03	6.85e-05	1.37e-06	3.37e-05	1.35e-05	9.00e-07
0.45000000	2.459603	1.02e-01	5.25e-02	3.42e-03	8.52e-05	1.70e-06	4.46e-05	1.73e-05	1.31e-06
0.50000000	2.718282	1.25e-01	6.43e-02	4.20e-03	1.05e-04	2.08e-06	5.74e-05	2.18e-05	1.79e-06
0.55000000	3.004166	1.51e-01	7.80e-02	5.11e-03	1.27e-04	2.53e-06	7.24e-05	2.70e-05	2.36e-06
0.60000000	3.320117	1.82e-01	9.38e-02	6.16e-03	1.53e-04	3.05e-06	8.99e-05	3.31e-05	3.03e-06
0.65000000	3.669297	2.17e-01	1.12e-01	7.37e-03	1.83e-04	3.66e-06	1.10e-04	4.02e-05	3.82e-06
0.70000000	4.055200	2.58e-01	1.33e-01	8.77e-03	2.18e-04	4.35e-06	1.34e-04	4.84e-05	4.73e-06
0.75000000	4.481689	3.04e-01	1.57e-01	1.04e-02	2.59e-04	5.15e-06	1.62e-04	5.79e-05	5.80e-06
0.80000000	4.953032	3.58e-01	1.85e-01	1.22e-02	3.05e-04	6.08e-06	1.93e-04	6.89e-05	7.04e-06
0.85000000	5.473947	4.19e-01	2.17e-01	1.44e-02	3.58e-04	7.14e-06	2.30e-04	8.15e-05	8.48e-06
0.90000000	6.049647	4.90e-01	2.53e-01	1.68e-02	4.19e-04	8.35e-06	2.72e-04	9.61e-05	1.01e-05
0.95000000	6.685894	5.70e-01	2.94e-01	1.96e-02	4.89e-04	9.74e-06	3.21e-04	1.13e-04	1.21e-05
1.00000000	7.389056	6.62e-01	3.42e-01	2.28e-02	5.68e-04	1.13e-05	3.77e-04	1.32e-04	1.43e-05
1.05000000	8.166170	7.66e-01	3.96e-01	2.65e-02	6.60e-04	1.31e-05	4.41e-04	1.54e-04	1.68e-05
1.10000000	9.025013	8.85e-01	4.57e-01	3.07e-02	7.64e-04	1.52e-05	5.14e-04	1.79e-04	1.97e-05
1.15000000	9.974182	1.02e+00	5.27e-01	3.54e-02	8.82e-04	1.76e-05	5.98e-04	2.07e-04	2.31e-05
1.20000000	11.023176	1.17e+00	6.06e-01	4.08e-02	1.02e-03	2.03e-05	6.94e-04	2.40e-04	2.69e-05
1.25000000	12.182494	1.35e+00	6.96e-01	4.70e-02	1.17e-03	2.34e-05	8.03e-04	2.77e-04	3.13e-05
1.30000000	13.463738	1.55e+00	7.98e-01	5.40e-02	1.35e-03	2.68e-05	9.28e-04	3.20e-04	3.63e-05
1.35000000	14.879732	1.77e+00	9.14e-01	6.20e-02	1.55e-03	3.08e-05	1.07e-03	3.68e-04	4.20e-05
1.40000000	16.444647	2.02e+00	1.05e+00	7.11e-02	1.77e-03	3.53e-05	1.23e-03	4.23e-04	4.85e-05
1.45000000	18.174145	2.31e+00	1.19e+00	8.13e-02	2.03e-03	4.04e-05	1.42e-03	4.85e-04	5.59e-05
1.50000000	20.085537	2.64e+00	1.36e+00	9.30e-02	2.32e-03	4.62e-05	1.62e-03	5.56e-04	6.43e-05
1.55000000	22.197951	3.00e+00	1.55e+00	1.06e-01	2.65e-03	5.28e-05	1.86e-03	6.36e-04	7.39e-05
1.60000000	24.532530	3.42e+00	1.77e+00	1.21e-01	3.02e-03	6.02e-05	2.13e-03	7.27e-04	8.48e-05
1.65000000	27.112639	3.89e+00	2.01e+00	1.38e-01	3.44e-03	6.86e-05	2.44e-03	8.31e-04	9.72e-05
1.70000000	29.964100	4.42e+00	2.28e+00	1.57e-01	3.92e-03	7.81e-05	2.78e-03	9.47e-04	1.11e-04
1.75000000	33.115452	5.01e+00	2.59e+00	1.79e-01	4.46e-03	8.89e-05	3.17e-03	1.08e-03	1.27e-04
1.80000000	36.598234	5.69e+00	2.94e+00	2.03e-01	5.07e-03	1.01e-04	3.62e-03	1.23e-03	1.45e-04
1.85000000	40.447304	6.44e+00	3.33e+00	2.31e-01	5.76e-03	1.15e-04	4.12e-03	1.40e-03	1.66e-04
1.90000000	44.701184	7.30e+00	3.77e+00	2.62e-01	6.53e-03	1.30e-04	4.68e-03	1.59e-03	1.89e-04
1.95000000	49.402449	8.26e+00	4.27e+00	2.97e-01	7.41e-03	1.48e-04	5.32e-03	1.80e-03	2.15e-04
2.00000000	54.598150	9.34e+00	4.82e+00	3.37e-01	8.40e-03	1.67e-04	6.05e-03	2.05e-03	2.44e-04

Problem B investigates the exponentially growing solution $y = e^{2t}$. This problem is particularly useful for studying error amplification because any numerical error introduced during the computation is magnified by the rapid growth of the exact solution. The results show that lower-order methods struggle significantly with this behavior. For $N=10$, Euler's maximum error reaches approximately 25.67, which is a substantial fraction of the exact solution value at $t = 2$. The second-order Taylor method reduces the maximum error to about 14.49, while the Midpoint method further lowers it to approximately 4.18. Increasing the number of steps improves the accuracy of all methods, but the higher-order methods remain substantially more accurate. For $N = 20$, RK4 achieves a maximum error of only 0.00246581, compared with 16.26055011 for Euler. At $N=40$, RK4's maximum error drops to approximately 1.67×10^{-4} , while Euler still exhibits an error greater than 9.33. Adams-Moulton and the Predictor-Corrector method also perform exceptionally well, maintaining errors on the order of 10^{-3} or smaller. The logarithmic error plots illustrate how truncation errors are amplified over time by the exponential growth of the solution. These results demonstrate the importance of both method order and stability when solving problems in which errors naturally grow throughout the integration interval.

Problem C

```

def f_3(t, y):
    #problem setting oscillation
    return -y**3 + np.cos(t) + np.sin(t)**3

def f_3_p(t, y):
    #first derivative
    return -np.sin(t) + 3*np.sin(t)**2 * np.cos(t) + (-3*y**2) * (-y**3 + np.cos(t) + np.sin(t)**3)

def y_exact_3(t):
    #exact solution
    return np.sin(t)

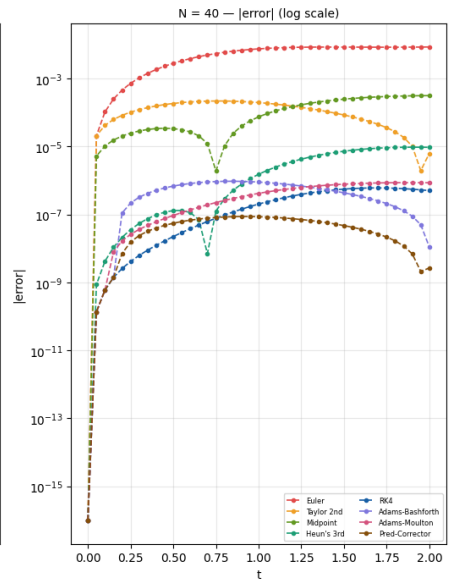
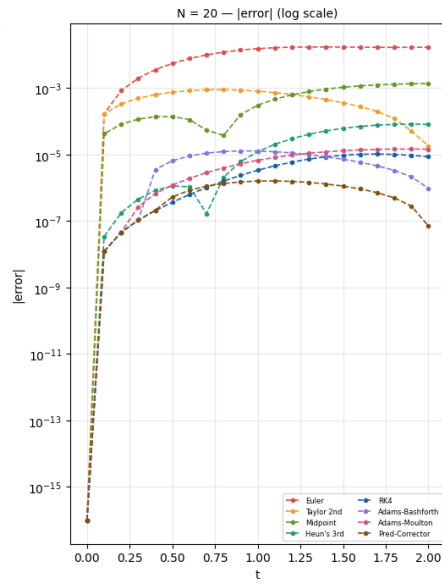
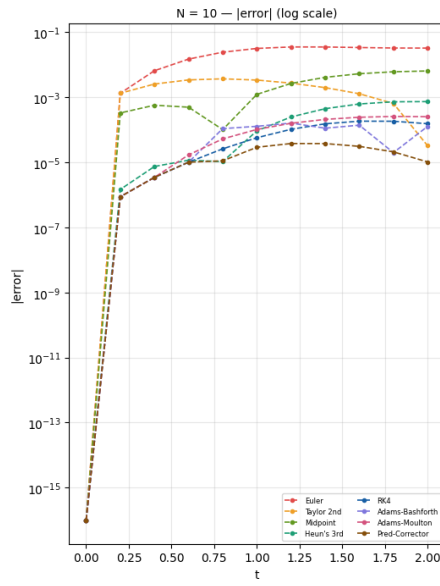
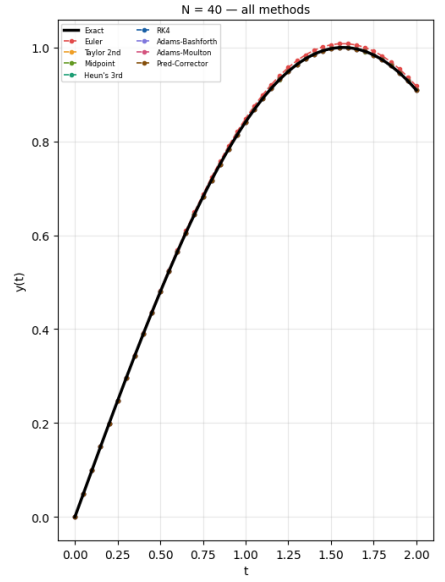
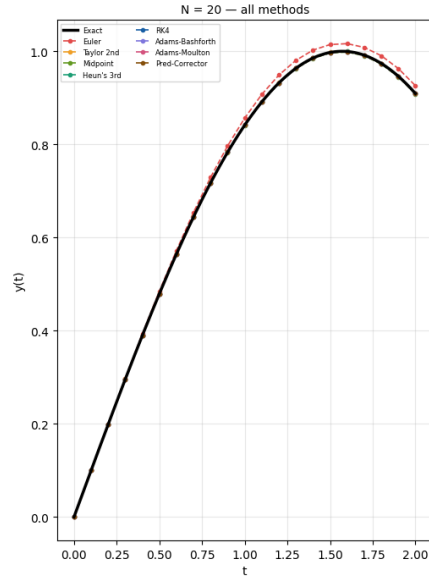
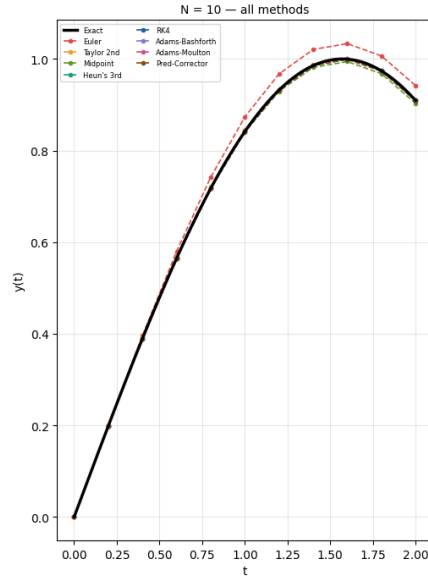
results_3 = {}
for N in N_s:
    results_3[N] = {
        'euler': euler(f_3, 0, 2, N, alpha=0),
        'taylor': taylor2(f_3, f_3_p, 0, 2, N, alpha=0),
        'midpoint': midpoint(f_3, 0, 2, N, alpha=0),
        'heuns': heuns(f_3, 0, 2, N, alpha=0),
        'RK4': RK4(f_3, 0, 2, N, alpha=0),
        'adams_b': adams_bashforth(f_3, 0, 2, N, alpha=0),
        'adams_m': adams_moulten(f_3, 0, 2, N, alpha=0),
        'pred_cor': predictor_corrector(f_3, 0, 2, N, alpha=0),
    }

#plotting
fig, axes = plt.subplots(2, 3, figsize=(16, 14))
for col, N in enumerate(N_s):
    t_fine = np.linspace(0, 2, 300)
    ax_top = axes[0][col]
    ax_top.plot(t_fine, y_exact_3(t_fine), 'k-', linewidth=2.5, label='Exact', zorder=10)
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_3[N][key]
        ax_top.plot(t_m, w_m, '--o', color=color, markersize=3, linewidth=1.2, label=label)
    ax_top.set_title(f'N = {N} - all methods', fontsize=10)
    ax_top.set_xlabel('t')
    ax_top.set_ylabel('y(t)')
    ax_top.legend(fontsize=6, ncol=2)
    ax_top.grid(True, alpha=0.3)

    ax_mid = axes[1][col]
    for key, label, color in zip(method_keys, method_labels, colors):
        t_m, w_m = results_3[N][key]
        err = np.abs(w_m - y_exact_3(t_m))
        ax_mid.semilogy(t_m, err + 1e-16, '--o', color=color, markersize=3, linewidth=1.2, label=label)
    ax_mid.set_title(f'N = {N} - |error| (log scale)', fontsize=10)
    ax_mid.set_xlabel('t')
    ax_mid.set_ylabel('|error|')
    ax_mid.legend(fontsize=6, ncol=2)
    ax_mid.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



```

#numerical tables
for N in N_s:

    t_vals = results_3[N]['euler'][0]    # all methods share the same t
    exact_vals_3 = y_exact_3(t_vals)

    data = {'t': t_vals, 'Exact': exact_vals_3}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        data[label] = w

    df = pd.DataFrame(data)

    max_row = {'t': 'max |error|', 'Exact': ''}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        max_row[label] = np.max(np.abs(w - y_exact_3(t_vals)))
    df = pd.concat([df, pd.DataFrame([max_row])], ignore_index=True)

    print(f'\n{"="*60}')
    print(f'  N = {N}   (h = {2/N:.4f})')
    print(f'{"="*60}')

    float_cols = [c for c in df.columns if c != 't']
    pd.set_option('display.float_format', '{:.8f}'.format)
    pd.set_option('display.max_columns', 20)
    pd.set_option('display.width', 220)
    print(df.to_string(index=False))

```

```
=====
N = 10 (h = 0.2000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
0.20000000	0.19866933	0.20000000	0.20000000	0.19899984	0.19866786	0.19866845	0.19866845	0.19866845	0.19866845
0.40000000	0.38941834	0.39598159	0.39197739	0.38998907	0.38941087	0.38941488	0.38941488	0.38941488	0.38941488
0.60000000	0.56464247	0.57958649	0.56806714	0.56513899	0.56463080	0.56463235	0.56463235	0.56465947	0.56463235
0.80000000	0.71735609	0.74171861	0.72105403	0.71725447	0.71736663	0.71733003	0.71724708	0.71740875	0.71736726
1.00000000	0.84147098	0.87327943	0.84485724	0.84023981	0.84156407	0.84141374	0.84134263	0.84157379	0.84150008
1.20000000	0.93203909	0.96730899	0.93476804	0.92936915	0.93228787	0.93193533	0.93187771	0.93219821	0.93207704
1.40000000	0.98544973	1.02069280	0.98743545	0.98133790	0.98589643	0.98529649	0.98533729	0.98565927	0.98548755
1.60000000	0.99957360	1.03340811	1.00085970	0.99426944	1.00019822	0.99938890	0.99943469	0.99981741	0.99960468
1.80000000	0.97384763	1.00659049	0.97449284	0.96775251	0.97457504	0.97366395	0.97382768	0.97410529	0.97386868
2.00000000	0.90929743	0.94188503	0.90932988	0.90284973	0.91003523	0.90914306	0.90917030	0.90955084	0.90930775
max error		0.03526991	0.00369794	0.00644770	0.00073780	0.00018470	0.00016138	0.00025766	0.00003795


```
=====
N = 20 (h = 0.1000)
=====
```

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
0.10000000	0.09983342	0.10000000	0.10000000	0.09987501	0.09983338	0.09983340	0.09983340	0.09983340	0.09983340
0.20000000	0.19866933	0.19949992	0.19900025	0.19875086	0.19866916	0.19866929	0.19866929	0.19866929	0.19866929
0.30000000	0.29552021	0.29749670	0.29600773	0.29563567	0.29551975	0.29552010	0.29552010	0.29552047	0.29552010
0.40000000	0.38941834	0.39297822	0.39004702	0.38955517	0.38941752	0.38941814	0.38941483	0.38941901	0.38941855
0.50000000	0.47942554	0.48492088	0.48017126	0.47956353	0.47942440	0.47942517	0.47941905	0.47942674	0.47942607
0.60000000	0.56464247	0.57229585	0.56547325	0.56475437	0.56464141	0.56464185	0.56463341	0.56464438	0.56464331
0.70000000	0.64421769	0.65408742	0.64509622	0.64427149	0.64421752	0.64421668	0.64420669	0.64422052	0.64421880
0.80000000	0.71735609	0.72932389	0.71824366	0.71731875	0.71735816	0.71735451	0.71734385	0.71736005	0.71735743
0.90000000	0.78332691	0.79711598	0.78418766	0.78316846	0.78333300	0.78332455	0.78331413	0.78333217	0.78332841
1.00000000	0.84147098	0.85669373	0.84227545	0.84116827	0.84148313	0.84146763	0.84145830	0.84147766	0.84147257
1.10000000	0.89120736	0.90743147	0.89193450	0.89074636	0.89122755	0.89120280	0.89119529	0.89121550	0.89120896
1.20000000	0.93203909	0.94885447	0.93267654	0.93141573	0.93206894	0.93203318	0.93202800	0.93204866	0.93204064
1.30000000	0.96355819	0.98062847	0.96410111	0.96277760	0.96359868	0.96355092	0.96354833	0.96356908	0.96355963
1.40000000	0.98544973	1.00253898	0.98589879	0.98452458	0.98550098	0.98544122	0.98544121	0.98546178	0.98545103
1.50000000	0.99749499	1.01447019	0.99785420	0.99644336	0.99755626	0.99748549	0.99748783	0.99750799	0.99749611
1.60000000	0.99957360	1.01639011	0.99984844	0.99841688	0.99964336	0.99956348	0.99956778	0.99958733	0.99957452
1.70000000	0.99166481	1.00834423	0.99186081	0.99042576	0.99174096	0.99165451	0.99166027	0.99167903	0.99166552
1.80000000	0.97384763	0.99045579	0.97396949	0.97254882	0.97392776	0.97383759	0.97384431	0.97386211	0.97384813
1.90000000	0.94630009	0.96292929	0.94635111	0.94496262	0.94638175	0.94629067	0.94629796	0.94631462	0.94630037
2.00000000	0.90929743	0.92605401	0.90927926	0.90793992	0.90937841	0.90928891	0.90929647	0.90931183	0.90929750
max error		0.01708925	0.00088757	0.00135751	0.00008167	0.00001030	0.00001278	0.00001453	0.00000160

=====

N = 40 (h = 0.0500)

=====

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000	0.00000000
0.05000000	0.04997917	0.05000000	0.05000000	0.04998438	0.04997917	0.04997917	0.04997917	0.04997917	0.04997917
0.10000000	0.09983342	0.09993751	0.09987502	0.09984380	0.09983341	0.09983342	0.09983342	0.09983342	0.09983342
0.15000000	0.14943813	0.14968756	0.14950035	0.14945358	0.14943812	0.14943813	0.14943813	0.14943814	0.14943813
0.20000000	0.19866933	0.19912527	0.19875183	0.19868961	0.19866931	0.19866933	0.19866922	0.19866935	0.19866934
0.25000000	0.24740396	0.24812590	0.24750621	0.24742868	0.24740392	0.24740396	0.24740374	0.24740399	0.24740397
0.30000000	0.29552021	0.29656487	0.29564146	0.29554879	0.29552015	0.29552020	0.29551989	0.29552024	0.29552023
0.35000000	0.34289781	0.34431796	0.34303705	0.34292944	0.34289773	0.34289780	0.34289739	0.34289786	0.34289784
0.40000000	0.38941834	0.39126145	0.38957429	0.38945198	0.38941825	0.38941833	0.38941783	0.38941840	0.38941838
0.45000000	0.43496553	0.43727237	0.43513664	0.43499990	0.43496542	0.43496552	0.43496493	0.43496561	0.43496558
0.50000000	0.47942554	0.48222891	0.47960999	0.47945914	0.47942541	0.47942552	0.47942486	0.47942563	0.47942559
0.55000000	0.52268723	0.52601082	0.52288301	0.52271836	0.52268710	0.52268720	0.52268648	0.52268734	0.52268729
0.60000000	0.56464247	0.56849998	0.56484736	0.56466927	0.56464236	0.56464244	0.56464166	0.56464261	0.56464254
0.65000000	0.60518641	0.60958102	0.60539805	0.60520691	0.60518634	0.60518636	0.60518554	0.60518657	0.60518648
0.70000000	0.64421769	0.64914202	0.64443368	0.64422989	0.64421769	0.64421763	0.64421678	0.64421788	0.64421776
0.75000000	0.68163876	0.68707523	0.68185669	0.68164066	0.68163888	0.68163868	0.68163783	0.68163898	0.68163884
0.80000000	0.71735609	0.72327775	0.71757361	0.71734579	0.71735638	0.71735599	0.71735515	0.71735635	0.71735617
0.85000000	0.75128041	0.75765221	0.75149528	0.75125615	0.75128091	0.75128029	0.75127946	0.75128070	0.75128049
0.90000000	0.78332691	0.79010732	0.78353711	0.78328714	0.78332770	0.78332677	0.78332597	0.78332724	0.78332700
0.95000000	0.81341550	0.82055833	0.81361923	0.81335892	0.81341663	0.81341533	0.81341458	0.81341588	0.81341559
1.00000000	0.84147098	0.84892734	0.84166667	0.84139657	0.84147250	0.84147078	0.84147009	0.84147140	0.84147107
1.05000000	0.86742323	0.87514347	0.86760959	0.86733026	0.86742520	0.86742299	0.86742236	0.86742368	0.86742331
1.10000000	0.89120736	0.89914291	0.89138341	0.89109544	0.89120984	0.89120709	0.89120654	0.89120786	0.89120744
1.15000000	0.91276394	0.92086885	0.91292895	0.91263297	0.91276697	0.91276363	0.91276316	0.91276448	0.91276402
1.20000000	0.93203909	0.94027133	0.93219257	0.93188924	0.93204271	0.93203874	0.93203835	0.93203967	0.93203916
1.25000000	0.94898462	0.95730701	0.94912632	0.94881634	0.94898885	0.94898423	0.94898393	0.94898524	0.94898469
1.30000000	0.96355819	0.97193901	0.96368804	0.96337214	0.96356304	0.96355776	0.96355755	0.96355884	0.96355825
1.35000000	0.97572336	0.98413660	0.97584145	0.97552042	0.97572884	0.97572289	0.97572277	0.97572405	0.97572342
1.40000000	0.98544973	0.99387509	0.98555628	0.98523091	0.98545583	0.98544923	0.98544920	0.98545045	0.98544979
1.45000000	0.99271299	1.00113563	0.99280830	0.99247944	0.99271968	0.99271246	0.99271251	0.99271374	0.99271304
1.50000000	0.99749499	1.00590512	0.99757941	0.99724793	0.99750223	0.99749443	0.99749455	0.99749576	0.99749503
1.55000000	0.99978376	1.00817616	0.99985770	0.99952448	0.99979151	0.99978319	0.99978338	0.99978456	0.99978381
1.60000000	0.99957360	1.00794700	0.99963744	0.99930340	0.99958181	0.99957301	0.99957326	0.99957442	0.99957364
1.65000000	0.99686503	1.00522154	0.99691916	0.99658523	0.99687362	0.99686443	0.99686473	0.99686586	0.99686506
1.70000000	0.99166481	1.00000939	0.99170960	0.99137670	0.99167373	0.99166421	0.99166456	0.99166565	0.99166484
1.75000000	0.98398595	0.99232587	0.98402172	0.98369078	0.98399512	0.98398535	0.98398574	0.98398680	0.98398597
1.80000000	0.97384763	0.98219203	0.97387466	0.97354663	0.97385699	0.97384704	0.97384746	0.97384849	0.97384765
1.85000000	0.96127520	0.96963468	0.96129372	0.96096950	0.96128468	0.96127463	0.96127508	0.96127607	0.96127521
1.90000000	0.94630009	0.95468638	0.94631026	0.94599076	0.94630962	0.94629954	0.94630000	0.94630095	0.94630009
1.95000000	0.92895972	0.93738542	0.92896166	0.92864775	0.92896924	0.92895919	0.92895967	0.92896058	0.92895972
2.00000000	0.90929743	0.91777579	0.90929119	0.90898372	0.90930688	0.90929693	0.90929742	0.90929828	0.90929742
max error		0.00847836	0.00021793	0.00031370	0.00000953	0.00000060	0.00000095	0.00000086	0.00000009

```

#error tables
method_keys = ['euler', 'taylor', 'midpoint', 'heuns', 'RK4', 'adams_b', 'adams_m', 'pred_cor']
method_labels = ['Euler', 'Taylor 2nd', 'Midpoint', "Heun's 3rd", 'RK4',
                 'Adams-Bashforth', 'Adams-Moulton', 'Pred-Corrector']

for N in N_s:
    h = 2 / N
    t_vals = results_3[N]['euler'][0]
    exact_vals_3 = y_exact_3(t_vals)

    data = {'t': np.round(t_vals, 4), 'Exact': exact_vals_3}
    for key, label in zip(method_keys, method_labels):
        _, w = results_3[N][key]
        data[label] = np.abs(w - exact_vals_3)

    df = pd.DataFrame(data)

    # format: exact in regular float, errors in scientific notation
    fmt = {'Exact': '{:.6f}'.format}
    for label in method_labels:
        fmt[label] = '{:.2e}'.format

    print(f'\nError table | h = {h} (N = {N})\n')
    print(df.to_string(index=False, formatters=fmt))
    print()

```

Error table | $h = 0.2$ ($N = 10$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.20000000	0.198669	1.33e-03	1.33e-03	3.31e-04	1.47e-06	8.78e-07	8.78e-07	8.78e-07	8.78e-07
0.40000000	0.389418	6.56e-03	2.56e-03	5.71e-04	7.47e-06	3.46e-06	3.46e-06	3.46e-06	3.46e-06
0.60000000	0.564642	1.49e-02	3.42e-03	4.97e-04	1.17e-05	1.01e-05	1.01e-05	1.70e-05	1.01e-05
0.80000000	0.717356	2.44e-02	3.70e-03	1.02e-04	1.05e-05	2.61e-05	1.09e-04	5.27e-05	1.12e-05
1.00000000	0.841471	3.18e-02	3.39e-03	1.23e-03	9.31e-05	5.72e-05	1.28e-04	1.03e-04	2.91e-05
1.20000000	0.932039	3.53e-02	2.73e-03	2.67e-03	2.49e-04	1.04e-04	1.61e-04	1.59e-04	3.80e-05
1.40000000	0.985450	3.52e-02	1.99e-03	4.11e-03	4.47e-04	1.53e-04	1.12e-04	2.10e-04	3.78e-05
1.60000000	0.999574	3.38e-02	1.29e-03	5.30e-03	6.25e-04	1.85e-04	1.39e-04	2.44e-04	3.11e-05
1.80000000	0.973848	3.27e-02	6.45e-04	6.10e-03	7.27e-04	1.84e-04	2.00e-05	2.58e-04	2.10e-05
2.00000000	0.909297	3.26e-02	3.25e-05	6.45e-03	7.38e-04	1.54e-04	1.27e-04	2.53e-04	1.03e-05

Error table | $h = 0.1$ ($N = 20$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.10000000	0.099833	1.67e-04	1.67e-04	4.16e-05	3.39e-08	1.21e-08	1.21e-08	1.21e-08	1.21e-08
0.20000000	0.198669	8.31e-04	3.31e-04	8.15e-05	1.76e-07	4.56e-08	4.56e-08	4.56e-08	4.56e-08
0.30000000	0.295520	1.98e-03	4.88e-04	1.15e-04	4.54e-07	1.05e-07	1.05e-07	2.67e-07	1.05e-07
0.40000000	0.389418	3.56e-03	6.29e-04	1.37e-04	8.27e-07	2.05e-07	3.52e-06	6.65e-07	2.11e-07
0.50000000	0.479426	5.50e-03	7.46e-04	1.38e-04	1.13e-06	3.65e-07	6.49e-06	1.20e-06	5.30e-07
0.60000000	0.564642	7.65e-03	8.31e-04	1.12e-04	1.06e-06	6.21e-07	9.06e-06	1.91e-06	8.36e-07
0.70000000	0.644218	9.87e-03	8.79e-04	5.38e-05	1.66e-07	1.01e-06	1.10e-05	2.83e-06	1.11e-06
0.80000000	0.717356	1.20e-02	8.88e-04	3.73e-05	2.07e-06	1.58e-06	1.22e-05	3.96e-06	1.34e-06
0.90000000	0.783327	1.38e-02	8.61e-04	1.58e-04	6.09e-06	2.36e-06	1.28e-05	5.26e-06	1.50e-06
1.00000000	0.841471	1.52e-02	8.04e-04	3.03e-04	1.21e-05	3.36e-06	1.27e-05	6.68e-06	1.58e-06
1.10000000	0.891207	1.62e-02	7.27e-04	4.61e-04	2.02e-05	4.56e-06	1.21e-05	8.14e-06	1.60e-06
1.20000000	0.932039	1.68e-02	6.37e-04	6.23e-04	2.99e-05	5.90e-06	1.11e-05	9.57e-06	1.55e-06
1.30000000	0.963558	1.71e-02	5.43e-04	7.81e-04	4.05e-05	7.26e-06	9.86e-06	1.09e-05	1.45e-06
1.40000000	0.985450	1.71e-02	4.49e-04	9.25e-04	5.13e-05	8.51e-06	8.52e-06	1.21e-05	1.30e-06
1.50000000	0.997495	1.70e-02	3.59e-04	1.05e-03	6.13e-05	9.50e-06	7.16e-06	1.30e-05	1.12e-06
1.60000000	0.999574	1.68e-02	2.75e-04	1.16e-03	6.98e-05	1.01e-05	5.82e-06	1.37e-05	9.20e-07
1.70000000	0.991665	1.67e-02	1.96e-04	1.24e-03	7.61e-05	1.03e-05	4.54e-06	1.42e-05	7.10e-07
1.80000000	0.973848	1.66e-02	1.22e-04	1.30e-03	8.01e-05	1.00e-05	3.32e-06	1.45e-05	4.95e-07
1.90000000	0.946300	1.66e-02	5.10e-05	1.34e-03	8.17e-05	9.42e-06	2.13e-06	1.45e-05	2.81e-07
2.00000000	0.909297	1.68e-02	1.82e-05	1.36e-03	8.10e-05	8.52e-06	9.55e-07	1.44e-05	7.02e-08

Error table | $h = 0.05$ ($N = 40$)

t	Exact	Euler	Taylor 2nd	Midpoint	Heun's 3rd	RK4	Adams-Bashforth	Adams-Moulton	Pred-Corrector
0.00000000	0.000000	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.00e+00
0.05000000	0.049979	2.08e-05	2.08e-05	5.21e-06	8.68e-10	1.35e-10	1.35e-10	1.35e-10	1.35e-10
0.10000000	0.099833	1.04e-04	4.16e-05	1.04e-05	4.18e-09	5.97e-10	5.97e-10	5.97e-10	5.97e-10
0.15000000	0.149438	2.49e-04	6.22e-05	1.54e-05	1.07e-08	1.39e-09	1.39e-09	7.83e-09	1.39e-09
0.20000000	0.198669	4.56e-04	8.25e-05	2.03e-05	2.12e-08	2.55e-09	1.10e-07	1.66e-08	7.02e-09
0.25000000	0.247404	7.22e-04	1.02e-04	2.47e-05	3.57e-08	4.13e-09	2.16e-07	2.59e-08	1.54e-08
0.30000000	0.295520	1.04e-03	1.21e-04	2.86e-05	5.40e-08	6.20e-09	3.20e-07	3.62e-08	2.38e-08
0.35000000	0.342898	1.42e-03	1.39e-04	3.16e-05	7.52e-08	8.88e-09	4.20e-07	4.77e-08	3.20e-08
0.40000000	0.389418	1.84e-03	1.56e-04	3.36e-05	9.71e-08	1.23e-08	5.14e-07	6.08e-08	4.01e-08
0.45000000	0.434966	2.31e-03	1.71e-04	3.44e-05	1.17e-07	1.67e-08	6.02e-07	7.59e-08	4.78e-08
0.50000000	0.479426	2.80e-03	1.84e-04	3.36e-05	1.30e-07	2.23e-08	6.83e-07	9.34e-08	5.50e-08
0.55000000	0.522687	3.32e-03	1.96e-04	3.11e-05	1.31e-07	2.93e-08	7.54e-07	1.14e-07	6.18e-08
0.60000000	0.564642	3.86e-03	2.05e-04	2.68e-05	1.13e-07	3.80e-08	8.14e-07	1.37e-07	6.78e-08
0.65000000	0.605186	4.39e-03	2.12e-04	2.05e-05	6.97e-08	4.88e-08	8.64e-07	1.62e-07	7.32e-08
0.70000000	0.644218	4.92e-03	2.16e-04	1.22e-05	7.00e-09	6.18e-08	9.02e-07	1.92e-07	7.77e-08
0.75000000	0.681639	5.44e-03	2.18e-04	1.90e-06	1.25e-07	7.75e-08	9.29e-07	2.24e-07	8.13e-08
0.80000000	0.717356	5.92e-03	2.18e-04	1.03e-05	2.90e-07	9.61e-08	9.43e-07	2.58e-07	8.40e-08
0.85000000	0.751280	6.37e-03	2.15e-04	2.43e-05	5.09e-07	1.18e-07	9.47e-07	2.95e-07	8.58e-08
0.90000000	0.783327	6.78e-03	2.10e-04	3.98e-05	7.86e-07	1.43e-07	9.39e-07	3.35e-07	8.66e-08
0.95000000	0.813416	7.14e-03	2.04e-04	5.66e-05	1.12e-06	1.71e-07	9.22e-07	3.75e-07	8.65e-08
1.00000000	0.841471	7.46e-03	1.96e-04	7.44e-05	1.52e-06	2.02e-07	8.97e-07	4.17e-07	8.56e-08
1.05000000	0.867423	7.72e-03	1.86e-04	9.30e-05	1.97e-06	2.36e-07	8.64e-07	4.59e-07	8.39e-08
1.10000000	0.891207	7.94e-03	1.76e-04	1.12e-04	2.48e-06	2.72e-07	8.25e-07	5.01e-07	8.14e-08
1.15000000	0.912764	8.10e-03	1.65e-04	1.31e-04	3.03e-06	3.11e-07	7.81e-07	5.43e-07	7.84e-08
1.20000000	0.932039	8.23e-03	1.53e-04	1.50e-04	3.62e-06	3.50e-07	7.34e-07	5.83e-07	7.48e-08
1.25000000	0.948985	8.32e-03	1.42e-04	1.68e-04	4.23e-06	3.90e-07	6.85e-07	6.22e-07	7.08e-08
1.30000000	0.963558	8.38e-03	1.30e-04	1.86e-04	4.86e-06	4.28e-07	6.34e-07	6.58e-07	6.64e-08
1.35000000	0.975723	8.41e-03	1.18e-04	2.03e-04	5.48e-06	4.66e-07	5.83e-07	6.92e-07	6.18e-08
1.40000000	0.985450	8.43e-03	1.07e-04	2.19e-04	6.10e-06	5.00e-07	5.32e-07	7.23e-07	5.70e-08
1.45000000	0.992713	8.42e-03	9.53e-05	2.34e-04	6.69e-06	5.30e-07	4.82e-07	7.51e-07	5.20e-08
1.50000000	0.997495	8.41e-03	8.44e-05	2.47e-04	7.24e-06	5.56e-07	4.33e-07	7.76e-07	4.69e-08
1.55000000	0.999784	8.39e-03	7.39e-05	2.59e-04	7.75e-06	5.77e-07	3.85e-07	7.98e-07	4.18e-08
1.60000000	0.999574	8.37e-03	6.38e-05	2.70e-04	8.20e-06	5.92e-07	3.39e-07	8.17e-07	3.67e-08
1.65000000	0.996865	8.36e-03	5.41e-05	2.80e-04	8.59e-06	6.00e-07	2.94e-07	8.32e-07	3.16e-08
1.70000000	0.991665	8.34e-03	4.48e-05	2.88e-04	8.92e-06	6.02e-07	2.51e-07	8.44e-07	2.66e-08
1.75000000	0.983986	8.34e-03	3.58e-05	2.95e-04	9.18e-06	5.98e-07	2.09e-07	8.53e-07	2.16e-08
1.80000000	0.973848	8.34e-03	2.70e-05	3.01e-04	9.36e-06	5.88e-07	1.68e-07	8.59e-07	1.66e-08
1.85000000	0.961275	8.36e-03	1.85e-05	3.06e-04	9.48e-06	5.73e-07	1.28e-07	8.62e-07	1.17e-08
1.90000000	0.946300	8.39e-03	1.02e-05	3.09e-04	9.53e-06	5.53e-07	8.87e-08	8.62e-07	6.89e-09
1.95000000	0.928960	8.43e-03	1.94e-06	3.12e-04	9.52e-06	5.28e-07	4.98e-08	8.60e-07	2.10e-09
2.00000000	0.909297	8.48e-03	6.24e-06	3.14e-04	9.45e-06	5.01e-07	1.10e-08	8.56e-07	2.66e-09

Problem C uses the nonlinear differential equation $y' = -y^3 + \cos(t) + \sin^3(t)$ with exact solution $y(t) = \sin(t)$. This problem was selected because it contains a nonlinear term while still having a known analytical solution for comparison. The cubic damping term $-y^3$ helps suppress excessive growth, making the problem more stable than the exponential-growth problem studied in Problem B. As expected, all methods perform better in this setting. For $N = 10$, Euler's error near the final time remains on the order of 10^{-2} , while Taylor's method reduces the error to approximately 10^{-3} . The higher-order methods perform significantly better, with RK4, Adams-Moulton, and the Predictor-Corrector method producing errors between 10^{-5} and 10^{-7} across most of the interval. As the step size is reduced from $h = 0.2$ to $h = 0.05$, all methods exhibit the expected decrease in error, with higher-order methods converging much more rapidly than Euler's method. By $N = 40$, RK4 and the Predictor-Corrector method maintain errors that are extremely small and nearly indistinguishable from the exact solution on the plots. These results show that nonlinear terms do not necessarily create numerical difficulties when stabilizing effects are present. The observed behavior agrees closely with theoretical expectations regarding convergence order, error propagation, and numerical stability, with RK4 and the corrected multistep methods providing the best balance of accuracy and efficiency.

Error Analysis:

Across all three problems, the observed errors agree with theoretical predictions. Euler (order 1) converges slowest, Taylor and Midpoint (order 2) converge moderately, Heun's (order 3) is better still, and RK4 and multi-step methods (order 4) are consistently the most accurate. Methods with the same formal order can still differ due to their leading error constants — for example, Adams-Moulton outperforms Adams-Bashforth because its implicit corrector step reduces the error constant. All methods remained stable within the step sizes tested, though Euler and Taylor would be first to lose stability if h were increased. Error propagation was well-behaved across all problems, with the nonlinear damping in Problem C actually helping to suppress error growth.

Conclusion:

Higher-order methods achieve dramatically better accuracy for the same computational cost. RK4 strikes the best balance for single-step integration, self-starting, stable, and 4th-order accurate. Multi-step methods match RK4's accuracy with fewer evaluations per step, but require carefulness and are more sensitive to initialization errors. The Predictor-Corrector is the most sophisticated method studied, combining explicit and implicit steps to achieve near-implicit accuracy without a nonlinear solve. Correct problem setup — particularly initial conditions — proved just as important as method choice, as an incorrect initial condition rendered all methods inaccurate regardless of their order.